

Beyond Pass/Fail

A Feature-Scale Benchmark for Correctness, Objective Code Quality, and Human AI-Assisted Development

Project overview

Software-engineering benchmarks have become a primary way to measure the capabilities of coding models and agents. Most established benchmarks, however, concentrate on whether a generated solution passes a test suite. This is essential, but incomplete: two implementations can satisfy the same behavioral requirements while differing substantially in complexity, maintainability, duplication, and the effort required for a human developer to understand and verify them.

We propose a benchmark for evaluating AI-generated software across three complementary dimensions:

1. **Behavioral correctness:** Does the implementation satisfy the feature’s requirements and acceptance criteria?
2. **Objective code quality:** What measurable structural and lexical properties characterize the implementation?
3. **Development provenance and human AI-usage behavior:** How did the project reach its current state, and how did real developers use AI assistance along the way?

The benchmark targets feature-scale software development. A representative task is not an isolated one-line repair, but a request such as “implement user login,” accompanied by user stories, functional requirements, and acceptance criteria. Tasks are manually authored from students’ own software projects, preserving genuine product intent rather than reconstructing intent solely from an issue and its reference patch.

Why another software-engineering benchmark?

Recent benchmarks have expanded well beyond narrow issue resolution. ProgramBench evaluates whether agents can reconstruct complete programs from an executable and documentation, leaving architecture and implementation entirely open. Senior SWE-Bench evaluates long-horizon features, migrations, bugs, and performance work from realistically underspecified requests. Its scoring combines runtime verifiers, an adaptive validation agent, task-specific rubric judging, and a model-based “taste” judge calibrated against expert review.

These approaches expose important frontier capabilities, but leave room for a complementary evaluation regime. ProgramBench primarily measures behavioral equivalence and reports selected structural differences between model and reference implementations. Senior SWE-Bench explicitly evaluates engineering taste and codebase fit, but important quality dimensions are mediated by model-based judgments and comparison with a reference patch.

Our benchmark instead asks whether implementation quality can be measured through a deterministic, independently reproducible static-analysis pipeline. It does not claim that static metrics fully capture engineering taste. Rather, it separates measurable code properties from subjective review so researchers can inspect both correctness and quality without collapsing them into a single opaque score.

Benchmark design

Each task package contains a project snapshot, a feature request, user stories or requirements, acceptance criteria, an executable evaluation harness, and a sanitized trace of the prior human–AI development process. The trace gives the coding agent more than a static repository snapshot: it provides context about the decisions, attempts, verification steps, and failures that produced the current code state. Two repositories may look identical at execution time while carrying very different histories; those histories can affect what a capable agent should inspect, preserve, question, or change.

Model outputs are retained as complete repositories or patches together with generation metadata. This supports both task-level scoring and analysis across models, projects, and task characteristics.

The first benchmark iteration uses a controlled, context-conditioned single-shot generation protocol: one fixed prompt, repository state, and prior-development trace produce one attempted implementation. Models receive the same task information and history, and the generation budget is held constant as far as provider interfaces permit. This protocol is deliberately narrower than a long-running agent evaluation. The mismatch between a feature-scale task and a single-shot attempt is treated as an explicit experimental variable, not hidden as an implementation detail.

Later iterations will introduce multi-turn and agentic conditions, including repository exploration, test execution, revision, and recovery from failures. Comparing these conditions will help isolate how much performance comes from the model, the interaction budget, and the surrounding agent scaffold.

Three-dimensional evaluation

1. Correctness

Solutions are executed against task-specific tests derived from the stated requirements and acceptance criteria. Correctness remains a necessary gate: code quality does not compensate for a feature that fails to meet its behavioral contract. Results retain granular test outcomes rather than only a final pass/fail label, allowing partial progress and failure patterns to be studied.

2. Objective code quality

Solutions are analyzed with `ts-repo-metrics`, a TypeScript and TSX repository-analysis engine. Its core syntax-aware analyses use Tree-sitter, supplemented by specialized tools where appropriate. Reported measures include:

- per-function cyclomatic and cognitive complexity;
- Halstead volume and related lexical measures;
- per-function GRAD-AI maintainability index;
- function length, nesting depth, and parameter counts;
- repository-level maintainability and complexity distributions;
- long functions, deep nesting, empty catches, and other code smells;
- duplicated code measured with `jscpd`; and
- optional React/TSX indicators such as JSX depth, hook usage, and monolithic component rate.

The engine emits versioned JSON reports, making every score traceable to a defined formula, analyzer version, repository commit, and analysis timestamp. The benchmark will publish metric definitions and aggregation rules so results can be reproduced without an LLM judge.

These metrics are reported as a profile rather than treated as interchangeable. For example, cyclomatic complexity measures control-flow structure, while cognitive complexity emphasizes nesting and comprehension burden. Maintaining the individual signals avoids concealing trade-offs behind a single composite score.

3. Development provenance and human AI-usage behavior

The benchmark incorporates anonymized AI-usage traces from students who used coding assistants while developing their projects. These traces have two roles. First, they are part of the task context: they let an evaluated coding agent reason not only about the code as it exists, but also about how the project arrived there. Second, they are research data from which we derive interaction frequency, session behavior, token usage where available, prompting patterns, exploration, code generation, verification or execution activity, and review habits.

This history-aware task formulation more closely reflects real software development. Engineers rarely encounter a codebase as an isolated artifact: they inherit design discussions, abandoned approaches, prior tool outputs, test failures, and evidence about why the current implementation looks the way it does. Supplying that provenance allows the benchmark to test whether agents can use process history rather than repeatedly rediscovering it from the repository.

The traces also provide a matched human-development comparison that model-only benchmarks generally lack. They allow questions such as whether successful developers spend more of their interaction budget on verification, whether higher interaction volume is associated with better code quality, whether provenance context improves an agent’s decisions, and whether one-shot model outputs exhibit different complexity or duplication patterns from AI-assisted human implementations.

Human traces are not treated as an unquestioned gold standard. Students vary in experience, workflow, and outcomes. The traces instead provide an empirical baseline for how real developers used AI under authentic project conditions. Released results should use anonymized or aggregated features, with raw conversational content governed separately to protect

student privacy.

Research questions

The benchmark is designed to support four initial questions:

1. How strongly does behavioral correctness correlate with objective maintainability and complexity?
2. Do models that pass the same acceptance tests produce meaningfully different quality profiles?
3. Does access to prior human–AI development history improve an agent’s correctness, efficiency, or code quality relative to receiving only the code snapshot?
4. How do single-shot model implementations differ from AI-assisted human implementations of feature-scale work?
5. How does moving from single-shot generation to multi-turn agency change correctness, code quality, cost, and verification behavior?

Contribution and relevance

The intended contribution is not another pass-rate leaderboard alone. It is an evaluation framework that joins specification-level feature intent, executable correctness, reproducible static quality measurement, and the provenance of human–AI development.

This framing complements ProgramBench’s study of whole-program reconstruction and Senior SWE-Bench’s study of senior-level correctness and taste. It addresses two under-measured questions: when an AI system produces a feature that appears to work, what kind of software did it actually leave behind, and can an agent perform better when it understands not only the current code but the development process that created it?

Current status and next steps

The static-analysis engine and its dashboard are implemented and publicly available at github.com/scottyUX/ts-repo-metrics. The initial benchmark protocol, task corpus, model set, and reporting schema are being finalized. The first release will establish the single-shot baseline; subsequent work will add multi-turn agent trajectories, broader language support, and longitudinal analysis of human AI-assisted development.

Selected references

- Yang, J. et al. (2026). *ProgramBench: Can Language Models Rebuild Programs From Scratch?* arXiv:2605.03546.
- Snorkel AI et al. (2026). *Senior SWE-Bench*. senior-swe-bench.snorkel.ai.
- Jimenez, C. E. et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*
- Gambo, I. et al. (2025). GRAD-AI maintainability methodology.